

• AVAILABLE FOR BACKEND WORK

Arash MohammadGholinejad

Backend Developer — C#/.NET & Python

Backend systems in two languages, one set of habits.

I like hard problems and I learn fast.

See projects

Get in touch

Email GitHub LinkedIn Website +98 939 213 5878 Military Service: Permanent Exemption Résumé

C# / ASP.NET Core

Web API — controllers, JWT auth

Application — MediatR (CQRS)

Domain — entities, invariants

Infrastructure — EF Core → SQL Server

Python / Django

DRF views — JWT auth, serializers

Services — business logic

Models — Django ORM

Data — PostgreSQL + Redis cache

// about

Building backends that hold up

I studied Computer Engineering and went into backend development because it's the work I actually enjoy doing every day: figuring out how a system should fit together and then making it hold up.

What keeps me interested is the hard part. I like problems without an obvious answer, and I'll stay with one until I understand it properly, not just until the error goes away. I learn quickly when a project needs something I don't know yet.

// where I've worked

Experience

- 

Software Engineer

Metable

Dec 21, 2024 - Jun 21, 2026

 - Developed the backend of a multi-tenant IoT telemetry platform: a REST API for users, projects, devices and telemetry data, alongside a message broker for device connectivity.
 - Built the device data ingestion pipeline with per-device authentication and topic-level access control, validating incoming messages and buffering them before batched writes to the database.
 - Implemented background processing for alert rules and notifications, offline-device detection and CSV export, and pushed live data to the web dashboard in real time.
 - Wrote unit and integration tests and containerized the services.
- 

Software Engineer

Hamkaran System

Jul 22, 2024 - Dec 20, 2024

 - Developed and maintained enterprise ERP applications on the in-house Rahkaran framework.
 - Worked within existing enterprise systems, implementing and maintaining business functionality.

C# · .NET
- 

Software Engineer

Alborz Afraz Tabarestan (Dornica)

Jul 23, 2023 - Jul 21, 2024

 - Developed and maintained backend services and RESTful APIs.
 - Designed and implemented an API Gateway service for centralized API management, routing and integration across services.
 - Built an online ArcGIS validation service and ArcGIS plugins for automated data/config validation and GIS workflows.
 - Refactored and modernized multiple projects for architecture, performance and maintainability.

Python · FastAPI · Celery
- 

Software Engineer

Zarebin (Hamrah Aval Internship)

Oct 2022 - Feb 2023

 - Developed and maintained backend services.
 - Designed and implemented RESTful APIs and managed database operations.

Python · Django
- 

Software Engineer

Jadooye Fekr (Internship)

Jun 2020 - Sep 2020

C# · ASP.NET · Web Forms

// toolbox

Skills

.NET / C#

- ASP.NET Core Web API
- MVC
- Entity Framework Core
- LINQ
- MediatR
- AutoMapper
- xUnit

Python

- FastAPI
- Django
- Flask
- DRF
- SQLAlchemy
- Pytest

Databases & caching

- SQL Server
- PostgreSQL
- MongoDB
- Redis

Tools & DevOps

- Docker
- Git
- GitHub Actions
- Swagger / OpenAPI

Architecture

- Clean Architecture
- DDD
- CQRS
- API Gateway
- Microservices

Practices

- OOP
- SOLID
- Design Patterns
- TDD
- Refactoring

// selected work

Projects

Four projects that show the same habits — layered architecture, JWT auth, Docker, CI — applied on both stacks.

GET

/catalog/movies

200OK

IMDB — movie & series API

IMDb-style catalog API on ASP.NET Core 8, built around Clean Architecture and tactical DDD so the domain enforces its own rules — aggregates and invariants — instead of leaning on validation at the API edge. CQRS via MediatR keeps reads and writes from tangling as the surface grew to 45 operations.

C#

ASP.NET Core

MediatR

EF Core

SQL Server

xUnit

Docker

View code on GitHub →

ProductKrait — e-commerce platform

Django e-commerce build where speed of delivery was the actual constraint, so the architecture stays straightforward on purpose: catalog, cart, orders and OTP-verified payments on Django's own ORM and auth conventions, with a JWT-secured REST API layered on top rather than replacing the server-rendered admin/staff dashboard.

Python

Django

DRF

JWT

Docker

[View code on GitHub →](#)[Live demo →](#)

AdderHub — social network

Instagram-style social app built with Django, where the interesting problem wasn't the feature list but serving it twice — a server-rendered site and a JWT-secured REST API — without duplicating the rules for what belongs in a feed. Redis sits in front of the one query that's read far more than it's written.

Python

Django

DRF

Redis

Docker

[View code on GitHub →](#)[Live demo →](#)

Store Management — retail API

Store REST API on ASP.NET Core 8, layered into Domain/Application/Infrastructure/Persistence so "can this order be placed" is a rule the domain checks once, not logic repeated in every place stock gets touched — orders are validated against stock and saved atomically, not checked-then-hoped-for.

C#

ASP.NET Core

EF Core

SQL Server

AutoMapper

Docker

[View code on GitHub →](#)

Auth Service (MongoDB)

JWT auth & user management on ASP.NET Core + MongoDB, PBKDF2 hashing, owner-or-admin authorization, Problem Details errors.

[Code →](#)

API Gateway & ArcGIS validation

Built at Dornica: centralized API routing/gateway and an automated ArcGIS config-validation service. Closed-source client work — described here, code not public.

[Details →](#)

Two projects, closer look

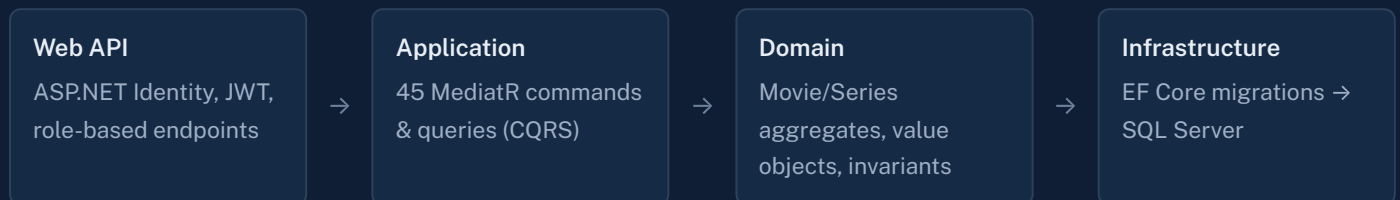
The résumé says what these do. This is closer to how — the layers, the calls I made on the way, and the pattern behind them.

IMDB — proving the domain layer earns its keep

Most of my day job is CRUD over an existing schema. IMDB was the project where I wanted a domain complex enough that Clean Architecture and DDD were the right tool, not decoration — a catalog where "what's a valid movie" is actually a rule, not just a database constraint.

[View code on GitHub →](#)

ARCHITECTURE



DECISIONS

CQRS over one service per controller

Splitting reads and writes into MediatR handlers kept 45 operations organized as the API grew, instead of a handful of bloated services.

Invariants on the aggregate, not the API

Validation lives inside the entities, so an illegal state is impossible to construct — not just rejected at the edge by a filter.

Tests aimed at behaviour, not endpoints

xUnit tests exercise domain rules and handlers directly, so a broken invariant fails fast without spinning up the whole API to notice.

THE PATTERN

A simplified version of the invariant-enforcing entities used across IMDB's aggregates:

```

public sealed class Movie : AggregateRoot
{
    private readonly List<CastMember> _cast = new();

    public static Result<Movie> Create(
        string title, DateOnly releaseDate, Rating rating)
    {
        if (releaseDate > DateOnly.FromDateTime(DateTime.UtcNow))
            return Result<Movie>.Fail(
                "Release date cannot be in the future.");

        return Result<Movie>.Ok(new Movie(title, releaseDate, rating));
    }

    public void AddCastMember(CastMember member)
    {
        if (_cast.Any(c => c.PersonId == member.PersonId))
            throw new DomainException(
                "This person is already cast in the movie.");

        _cast.Add(member);
        RaiseDomainEvent(new CastMemberAddedEvent(Id, member.PersonId));
    }
}

```

AdderHub — one codebase, two front doors

A social app usually needs a website and an API. AdderHub serves both — server-rendered pages for the site, a JWT-secured DRF API for anything else — from the same models and the same business logic, instead of maintaining two parallel implementations of "what's in my feed."

[View code on GitHub →](#)

ARCHITECTURE

Server-rendered views

Django templates, sessions

DRF API

JWT auth, Swagger/ReDoc

↓ both call the same service layer ↓

Services

Feed, posts, follows — one place decides the rules

↓

Data

DECISIONS

One codebase, two front doors

The website and the API call the same service functions, so a rule about the feed only has to be right once.

Redis in front of the feed, not the whole app

The following/explore feed is the most repeated read by far, so it's the one thing cached — not a blanket cache over everything.

UUID keys on public-facing models

Post and profile IDs don't leak how many users or posts exist, without touching anything else in the schema.

THE PATTERN

A simplified version of the shared service layer behind both front doors:

```
class FeedService:
    """Shared by the server-rendered view and the DRF
    endpoint -- one place decides what's in a feed."""

    def __init__(self, cache):
        self.cache = cache

    def get_following_feed(self, user, page=1):
        key = f"feed:{user.id}:{page}"
        if cached := self.cache.get(key):
            return cached

        posts = (
            Post.objects
            .filter(author__in=user.following.all())
            .select_related("author")
            .prefetch_related("likes", "comments")
            .order_by("-created_at")
        )
        page_obj = paginate(posts, page)
        self.cache.set(key, page_obj, timeout=60)
        return page_obj
```

// background

Education

B.S. in Computer Engineering

Babol Noshirvani University of Technology

2016 – 2021

Served as a teaching assistant for programming courses.

Machine Learning Specialization — Coursera, Feb 2024 · Certificate →

Deep Learning Specialization — Coursera, Mar 2024 · Certificate →

Let's talk about your backend.

Open to remote backend roles and freelance work in .NET or Python — API design, Clean Architecture refactors, or building something new from a spec.

arash.m.gholinejad@gmail.com

GITHUB	github.com/codesbygom
LINKEDIN	linkedin.com/in/arash1997
WEBSITE	codesbygom.github.io
PHONE	+98 939 213 5878
MILITARY SERVICE	Permanent Exemption
RÉSUMÉ	View & download (PDF)